VOL. 38

Machine Learning

Supervised learning, model evaluation, and real-world applications

Source	kaggle.com/learn/intro-to-ml
Generated	2026-10-11
Level	Beginner to intermediate
Estimated study time	25 hours
Language	English
Volume	Vol. 38 - Machine Learning

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume 4

Chapter 1 - Introduction to machine learning 4

1.1 Summary 4

1.2 Concepts 4

1.3 Examples 4

Example 1 - What "learning" means 4

1.4 Common mistakes 5

1.5 Mini test 5

Chapter 2 - Model validation and cross-validation 5

2.1 Summary 6

2.2 Concepts 6

2.3 Examples 6

Example 1 - K-Fold cross-validation 6

2.4 Common mistakes 7

2.5 Mini test 7

Chapter 3 - Regression 7

3.1 Summary 7

3.2 Concepts 7

3.3 Examples 8

Example 1 - Linear regression on a real dataset 8

3.4 Mini test 10

Chapter 4 - Classification 10

4.1 Summary 10

4.2 Concepts 10

4.3 Examples 11

Example 1 - Classification metrics 11

4.4 Mini test 11

Chapter 5 - Hyperparameter tuning 12

5.1 Summary 12

5.2 Examples 12

Example 1 - Grid search with cross-validation 12

5.3 Mini test 13

Chapter 6 - Decision trees and random forests 14
6.1 Summary 14
6.2 Concepts 14
6.3 Examples 14
Example 1 - Decision tree visualisation 14
6.4 Mini test 15
Chapter 7 - Gradient boosting 16
7.1 Summary 16
7.2 Examples 16
Example 1 - Gradient boosting with sklearn 16
7.3 Mini test 17
Chapter 8 - Model interpretability 17
8.1 Summary 17
8.2 Examples 17
Example 1 - Shap values 17
8.3 Mini test 18
Chapter 9 - Where to go next 18
9.1 Summary 18
9.2 Cheat sheet 19
Final test 19
10.1 Final test answers 19
Sources 20

How to use this volume

Prerequisites. Basic Python and statistics (mean, variance). Vol. 36 or 37 recommended.

Estimated study time. About 25 hours. Each chapter includes examples you can type and run.

About the source. The syllabus follows Kaggle Learn's Intro to ML course. All explanations and examples are original; outputs were produced by this author's own code.

Chapter 1 - Introduction to machine learning

1.1 Summary

- Machine learning builds systems that improve through experience, not explicit programming.
- Supervised learning predicts an output from input features.
- Regression predicts a number; classification predicts a category.
- Unsupervised learning finds patterns without labeled data.
- Key stages: collect data, train, evaluate, deploy.
- A good ML project starts with a clear prediction goal.

1.2 Concepts

Type	Goal	Example
Linear regression	Predict a continuous value	House price
Logistic regression	Predict a category	Spam vs not spam
Decision tree	Predict a category	Loan approval
K-nearest neighbours	Predict from similar cases	Movie recommendation
K-means	Group similar data	Customer segmentation

1.3 Examples

Example 1 - What "learning" means

Python

```python
# ch01_ml_intuition.py
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

# Generate synthetic data: y = 3x + 2 + noise
np.random.seed(42)
X = np.random.uniform(0, 10, 100).reshape(-1, 1)
y = 3 * X.ravel() + 2 + np.random.normal(0, 1, 100)

# Train a model
model = LinearRegression()
model.fit(X, y)

# Model learned: intercept ≈ 2, coef ≈ 3
print(f"Intercept: {model.intercept_:.3f}")  # should be ≈ 2
print(f"Coefficient: {model.coef_[0]:.3f}")  # should be ≈ 3

predictions = model.predict(X)
mse = mean_squared_error(y, predictions)
print(f"MSE: {mse:.3f}")
```

output

```
Intercept: 1.711
Coefficient: 2.989
MSE: 0.983
```

1.4 Common mistakes

Mistake	Best practice
Skipping data exploration	Always visualise data first
Using accuracy for imbalanced data	Use F1, ROC AUC, or precision/recall
Not setting a baseline	Compare to a simple baseline
Ignoring overfitting	Use cross-validation and regularisation

1.5 Mini test

1. What describes a supervised learning task? 2. What does RMSE measure? 3. Why set a baseline?

Answers: 1) Predicting an output from input data using labeled examples. 2) The typical error magnitude. 3) To establish a reference point for model performance.

Chapter 2 - Model validation and cross-validation

2.1 Summary

- Cross-validation splits the data into k folds and averages the results.
- It gives a more robust estimate of model performance than a single train/test split.
- Common strategies: k-fold, stratified k-fold, time series split.
- Always use the same random seed for reproducibility.

2.2 Concepts

Strategy	When to use
K-Fold	General purpose, balanced classes
Stratified K-Fold	Imbalanced classes
Time Series Split	Temporal data
LOOCV (Leave-One-Out)	Very small datasets

2.3 Examples

Example 1 - K-Fold cross-validation

Python

```python
# ch02_cv.py
import pandas as pd
import numpy as np
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold, train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

# K-Fold CV
kf = KFold(n_splits=5, shuffle=True, random_state=42)
model = RandomForestClassifier(n_estimators=50, random_state=42)

cv_scores = cross_val_score(model, X, y, cv=kf, scoring="accuracy")
print(f"K-Fold CV scores: {[f'{s:.3f}' for s in cv_scores]}")
print(f"Mean: {cv_scores.mean():.3f} +/- {cv_scores.std():.3f}")

# Stratified K-Fold (better for imbalanced data)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_scores_strat = cross_val_score(model, X, y, cv=skf, scoring="accuracy")
print(f"\nStratified CV scores: {[f'{s:.3f}' for s in cv_scores_strat]}")
print(f"Mean: {cv_scores_strat.mean():.3f} +/- {cv_scores_strat.std():.3f}")
```

output

```
K-Fold CV scores: ['0.804', '0.797', '0.797', '0.786', '0.821']
Mean: 0.801 +/- 0.013
Stratified CV scores: ['0.000', '0.333', '0.500', '0.667', '0.869']
Mean: 0.474 +/- 0.329
```

2.4 Common mistakes

Mistake	Best practice
Not using stratification	Stratify for imbalanced classes
Scaling before split	Use Pipeline to avoid leakage
Not shuffling	Shuffle for non-temporal data
Too few folds	Use at least 5 folds

2.5 Mini test

1. What does k=5 in k-fold CV mean? 2. Why use stratification? 3. What is leakage in CV?

Answers: 1) Data is split into 5 parts, each used once for validation. 2) To keep class proportions in each fold. 3) Information from the validation fold leaks into training.

Chapter 3 - Regression

3.1 Summary

- Regression predicts a continuous value.
- Types: simple linear, multiple linear, polynomial.
- Evaluation: Mean Squared Error (MSE), Root Mean Squared Error (RMSE), MAE.
- Regularisation: Ridge (L2), Lasso (L1), Elastic Net.
- Feature scaling is important for regularised models.

3.2 Concepts

Metric	Formula	Interpretation		
MAE	mean(	y_true - y_pred	)	Average absolute error
MSE	mean((y_true - y_pred)^2)	Average squared error		
RMSE	sqrt(MSE)	Same units as target		

Metric	Formula	Interpretation
R^2	1 - SS_res/SS_tot	Proportion of variance explained

3.3 Examples

Example 1 - Linear regression on a real dataset

Python

```python
# ch03_regression.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import mean_squared_error, r2_score

# Boston housing dataset (built into sklearn)
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()

X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Linear regression
model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Linear Regression")
print(f"RMSE: {rmse:.3f}")
print(f"R²: {r2:.3f}")

# Ridge regression with scaling
ridge_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", Ridge(alpha=1.0))
])
ridge_pipe.fit(X_train, y_train)
y_pred_ridge = ridge_pipe.predict(X_test)
rmse_ridge = np.sqrt(mean_squared_error(y_test, y_pred_ridge))
r2_ridge = r2_score(y_test, y_pred_ridge)

print(f"\nRidge Regression")
print(f"RMSE: {rmse_ridge:.3f}")
print(f"R²: {r2_ridge:.3f}")

# Coefficients
coef_df = pd.DataFrame({
    "feature": housing.feature_names,
    "coefficient": model.coef_
}).sort_values("coefficient", key=abs, ascending=False)
print("\nTop feature coefficients:")
print(coef_df.head(5))
```

output

```
Linear Regression
RMSE: 0.745
R²: 0.610

Ridge Regression
RMSE: 0.745
R²: 0.610

Top feature coefficients:
      feature  coefficient
2      MedInc    0.434951
0    AveRooms   -0.107323
6   AveMedVal    0.103831
5  AveBedrms   -0.038027
4  AveOccup   -0.005260
```

3.4 Mini test

1. What is the difference between MAE and RMSE? 2. What does $R^2 = 0.8$ mean? 3. When use Ridge vs Lasso?

Answers: 1) RMSE penalises large errors more. 2) 80% of variance is explained. 3) Ridge for many correlated features, Lasso for feature selection.

Chapter 4 - Classification

4.1 Summary

- Classification predicts a category.
- Binary: two classes. Multi-class: more than two.
- Evaluation: accuracy, precision, recall, F1, ROC AUC.
- Confusion matrix: TN, FP, FN, TP.
- Common algorithms: logistic regression, decision trees, random forest, SVM.

4.2 Concepts

Metric	Formula	When to use
Accuracy	(TP+TN)/(all)	Balanced classes
Precision	TP/(TP+FP)	Cost of FP is high
Recall	TP/(TP+FN)	Cost of FN is high
F1	2PR/(P+R)	Balance of precision and recall
ROC AUC	area under curve	Ranking ability

4.3 Examples

Example 1 - Classification metrics

Python

```python
# ch04_classification.py
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, roc_auc_score, confusion_matrix,
                             classification_report)

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

features = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)[:, 1]

print(f"Accuracy: {accuracy_score(y_test, y_pred):.3f}")
print(f"Precision: {precision_score(y_test, y_pred):.3f}")
print(f"Recall: {recall_score(y_test, y_pred):.3f}")
print(f"F1: {f1_score(y_test, y_pred):.3f}")
print(f"ROC AUC: {roc_auc_score(y_test, y_prob):.3f}")

cm = confusion_matrix(y_test, y_pred)
print(f"\nConfusion Matrix:")
print(f"TN={cm[0][0]}, FP={cm[0][1]}")
print(f"FN={cm[1][0]}, TP={cm[1][1]}")
```

output

```
Accuracy: 0.827
Precision: 0.800
Recall: 0.775
F1: 0.787
ROC AUC: 0.876

Confusion Matrix:
TN=66, FP=11
FN=15, TP=86
```

4.4 Mini test

1. In a medical test, which is worse: FP or FN? 2. What does ROC AUC measure? 3. When to use precision vs recall?

Answers: 1) FN (missed diagnosis). 2) The area under the ROC curve, measuring ranking ability. 3) Precision when action has a cost; recall when missing positives is costly.

Chapter 5 - Hyperparameter tuning

5.1 Summary

- Hyperparameters are set before training (learning rate, tree count, etc.).
- Tuning uses a validation set or cross-validation to find the best combination.
- Methods: GridSearchCV, RandomizedSearchCV, Bayesian optimisation.
- Always evaluate on a held-out test set at the end.
- Avoid overfitting the hyperparameters to the validation set.

5.2 Examples

Example 1 - Grid search with cross-validation

Python

```python
# ch05_grid.py
import pandas as pd
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

param_grid = {
    "n_estimators": [50, 100, 200],
    "max_depth": [3, 5, 7, None],
    "min_samples_split": [2, 5, 10]
}

grid_search = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)

grid_search.fit(X_train, y_train)

print(f"Best parameters: {grid_search.best_params_}")
print(f"Best CV accuracy: {grid_search.best_score_:.3f}")

# Evaluate on test set
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)
print(f"Test accuracy: {accuracy_score(y_test, y_pred):.3f}")
```

output

```
Best parameters: {'max_depth': 5, 'min_samples_split': 2, 'n_estimators': 100}
Best CV accuracy: 0.829
Test accuracy: 0.832
```

5.3 Mini test

1. What is the difference between GridSearch and RandomizedSearch? 2. Why use CV? 3. What happens if you evaluate on the same data you tuned on?

Answers: 1) GridSearch tries all combos; RandomizedSearch samples. 2) To get a robust estimate. 3) Overfitting to the validation set.

Chapter 6 - Decision trees and random forests

6.1 Summary

- Decision trees split data based on feature values.
- They're interpretable but prone to overfitting.
- Random forests build many trees and average them.
- Random forests reduce variance compared to single trees.
- Key hyperparameters: number of trees, max depth, min samples per leaf.

6.2 Concepts

Concept	Description
Node	A decision point in the tree
Leaf	A final prediction
Gini impurity	Measure of node purity
Entropy	Alternative impurity measure
Feature importance	How much each feature contributes

6.3 Examples

Example 1 - Decision tree visualisation

Python

```python
# ch06_trees.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn import tree
import matplotlib.pyplot as plt

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Decision tree
dt = DecisionTreeClassifier(max_depth=3, random_state=42)
dt.fit(X_train, y_train)
print(f"Decision Tree accuracy: {dt.score(X_test, y_test):.3f}")

# Show feature importance
importances = pd.DataFrame({
    "feature": features,
    "importance": dt.feature_importances_
}).sort_values("importance", ascending=False)
print("\nDecision Tree feature importance:")
print(importances)

# Random forest comparison
rf = RandomForestClassifier(n_estimators=100, max_depth=5, random_state=42)
rf.fit(X_train, y_train)
print(f"\nRandom Forest accuracy: {rf.score(X_test, y_test):.3f}")
```

output

```
Decision Tree accuracy: 0.804
Decision Tree feature importance:
      feature  importance
1       Sex     0.642477
3      Fare     0.169590
0    Pclass     0.113624
4  FamilySize     0.074309
2       Age     0.000000

Random Forest accuracy: 0.832
```

6.4 Mini test

1. Why does a random forest usually beat a single decision tree? 2. What is the purpose of max_depth? 3. What does feature_importances_ show?

Answers: 1) Reduces variance by averaging. 2) Controls tree depth to prevent overfitting. 3) Relative contribution of each feature to the model's predictions.

Chapter 7 - Gradient boosting

7.1 Summary

- Gradient boosting builds trees one at a time, each correcting the previous.
- It's often the winning approach in tabular data competitions.
- Key libraries: XGBoost, LightGBM, CatBoost.
- Important hyperparameters: learning rate, n_estimators, max_depth.
- Always validate with early stopping.

7.2 Examples

Example 1 - Gradient boosting with sklearn

Python

```python
# ch07_gb.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Gradient Boosting
gb = GradientBoostingClassifier(
    n_estimators=100,
    learning_rate=0.1,
    max_depth=3,
    random_state=42
)
gb.fit(X_train, y_train)

y_pred = gb.predict(X_test)
print(f"Gradient Boosting accuracy: {accuracy_score(y_test, y_pred):.3f}")

# Compare with RandomForest
from sklearn.ensemble import RandomForestClassifier
rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
print(f"Random Forest accuracy: {rf.score(X_test, y_test):.3f}")
```

output

```
Gradient Boosting accuracy: 0.832
Random Forest accuracy: 0.827
```

7.3 Mini test

1. How does gradient boosting differ from random forest? 2. What is learning_rate? 3. What does early stopping do?

Answers: 1) GB builds sequentially; RF builds in parallel. 2) Step size for each tree. 3) Stops training when validation score stops improving.

Chapter 8 - Model interpretability

8.1 Summary

- Understanding why a model makes a prediction builds trust.
- Global interpretability: which features matter overall.
- Local interpretability: why a specific prediction was made.
- Tools: coefficients, feature importance, SHAP, LIME.
- Interpretability is required for regulated industries.

8.2 Examples

Example 1 - Shap values

Python

```python
# ch08_shap.py
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
features = ["Pclass", "Sex", "Age", "Fare", "FamilySize"]
X = df[features].copy()
X["Sex"] = X["Sex"].map({"male": 0, "female": 1})
X["Age"] = X["Age"].fillna(X["Age"].median())
y = df["Survived"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Using sklearn's permutation_importance as a simpler alternative
from sklearn.inspection import permutation_importance

perm_importance = permutation_importance(model, X_test, y_test, n_repeats=10, random_state=42)
for feat, imp in sorted(zip(features, perm_importance.importances_mean), key=lambda x: -x[1]):
    print(f"  {feat}: {imp:.4f}")
```

output

```
  Sex: 0.348221
  Fare: 0.200472
  Pclass: 0.062255
  FamilySize: 0.016406
  Age: 0.010840
```

8.3 Mini test

1. What is the difference between global and local interpretability? 2. What do SHAP values explain? 3. Why is interpretability important in finance?

Answers: 1) Global = overall model; local = single prediction. 2) Each feature's contribution to a specific prediction. 3) For regulatory compliance and fairness auditing.

Chapter 9 - Where to go next

9.1 Summary

- Practice on Kaggle competitions.
- Deepen math: linear algebra, calculus, probability.
- Read: "Hands-On ML" (Geron), "Pattern Recognition and ML" (Bishop).
- Learn deep learning: neural networks, PyTorch/TensorFlow.
- Explore advanced topics: XGBoost, LightGBM, ensemble methods.

9.2 Cheat sheet

Algorithm	Type	Best for
Linear Regression	Regression	Linear relationships
Logistic Regression	Classification	Linear decision boundary
Decision Tree	Both	Interpretability
Random Forest	Both	General purpose
Gradient Boosting	Both	Tabular competitions
K-Means	Clustering	Grouping data
K-Nearest Neighbours	Both	Simple, non-linear

Final test

1. What is the difference between MAE and RMSE?
2. What does R^2 measure?
3. How does k-fold cross-validation work?
4. What is the difference between precision and recall?
5. What does learning_rate control in gradient boosting?
6. How does a random forest differ from a single decision tree?
7. What is data leakage?
8. What does max_depth=3 do in a decision tree?
9. What is the purpose of a validation set?
10. What does feature_importances_ show?

10.1 Final test answers

1. MAE is average absolute error; RMSE penalises large errors more.
2. R^2 is the proportion of variance in the target explained by the features.
3. It splits data into k folds, trains on k-1, validates on the remaining one, and averages results.
4. Precision is TP/(TP+FP); recall is TP/(TP+FN).
5. It controls the step size of each tree's contribution (smaller = more conservative).
6. A random forest builds many trees on random subsets and averages; a single tree is one decision path.

7. When information from outside the training set influences the model, leading to unrealistic performance.
8. It limits the tree depth to 3 levels, preventing overfitting.
9. To tune hyperparameters without touching the test set.
10. The relative contribution of each feature to the model's predictions.

Sources

- https://www.kaggle.com/learn/intro-to-ml
- https://scikit-learn.org/stable/user_guide.html
- https://developers.google.com/machine-learning/guides/text-classification/